

Infosys Springboard Virtual Internship 6.0

Completion Report

Contributor Detail

Batch Number : 2

Start date : 04/09/2025

Name : Monika N V

Internship Duration: 8 Weeks

1. Project Title

AI-Based Legal Reference and Case Retrieval System

2. Project Objective

The project “**The AI-Based Legal Reference and Case Retrieval System**” is an AI-powered assistant that helps users retrieve legal provisions, statutes, and case-law efficiently. Users can create accounts, login securely, and manage their profiles. The system stores conversation history in a SQLite database for each user session. The backend uses **LangChain** to manage AI workflows and a **RAG (Retrieval-Augmented Generation) chain** to fetch and summarize relevant legal documents from a vector database (Pinecone). The AI responses always include a legal disclaimer, relevant provisions, a concise summary, and citations. The user interface is built with **Streamlit**, featuring a dark theme, user-friendly layout, profile pictures, and easy navigation. Passwords are securely hashed, and all user data and chat history are safely stored.

3. Project description in detail

Project Title: The AI-Based Legal Reference and Case Retrieval System

Overview:

The AI-Based Legal Reference and Case Retrieval System is an AI-powered application designed to help users, including lawyers, students, and the general public, retrieve legal provisions, statutes, and case-law efficiently. This system uses advanced AI technologies to process queries, retrieve relevant documents, and provide summarized answers with precise legal references. The platform ensures faster, reliable, and context-aware responses, reducing the manual effort required in legal research.

Approach and Workflow:

1. User Interaction:

1. Users access the system through a **web interface built using Streamlit**.

2. Each user is authenticated and can maintain a session with chat history stored securely.

2. Query Processing and Retrieval:

1. User questions are passed to a **Retrieval-Augmented Generation (RAG) chain** implemented using **LangChain**.
2. The system uses **HuggingFace embeddings** (all-MiniLM-L6-v2) to convert user queries into semantic vectors.
3. **Pinecone vector database** stores the legal documents in vector format for fast semantic search.
4. A **history-aware retriever** ensures context from previous conversation turns is included when forming search queries.

3. Response Generation:

1. The retrieved documents are fed into a **GPT-based language model** (gpt-3.5-turbo) through LangChain.
2. The model generates answers while strictly adhering to the legal context from the retrieved documents.
3. Answers are structured with **legal disclaimer, relevant provisions, summaries, and citations**, ensuring clarity and accuracy.

4. Data Storage and Management:

1. **SQLite** is used to manage user authentication and store chat history.
2. This ensures session continuity, enabling the system to provide history-aware responses.

Technology Used:

Python – core programming language.

Streamlit – web application framework for interactive UI.

LangChain – for building RAG chains and context-aware document retrieval.

HuggingFace Embeddings – for semantic similarity search.

Pinecone Vector Database – high-performance vector search engine.

OpenAI GPT Models – to generate human-readable answers.

SQLite – to manage user data and chat history.

Environment Variables & dotenv – for secure API key management.

5. Impact and Real-World Implementation:

1. Enables faster and more accurate access to legal information compared to manual searches.
2. Supports research for lawyers, students, and legal professionals by providing **relevant statutes and case references** instantly.
3. Reduces time and effort in legal research and preliminary case study preparation.
4. Can be extended to include additional jurisdictions or updated legal content for broader applicability.
5. Ensures **legal compliance** by including mandatory disclaimers, thus clarifying the AI's advisory limitations.

The project demonstrates how AI and vector-based search technologies can be integrated into legal research systems. By combining LangChain with Pinecone and GPT models, it creates an intelligent, context-aware platform that provides precise legal references and summaries in real time. The system is scalable, secure, and can have practical applications in legal firms, research institutions, and educational environments.

4. Timeline Overview

Week	Activities Planned	Activities Completed
Week 1	Preparing python setup and git github setup	Completed on time
Week 2	Collections of documents and chunking them in chunks	Completed on time
Week 3	Embedding the files and perform crud in pine cone	Completed on time
Week 4	Creation of RAG model	Completed on time
Week 5	Connect RAG model to pinecone to get legal outputs	Completed on time
Week 6	Starting of making a	Completed on time
Week 7	Additional features history control, authentication, user database management added	Completed on time
Week 8	Creation of documentation	Completed on time

5a. Key Milestones

Milestone	Description	Date Achieved
Project Kickoff	Its starts with a basic idea about the project and the idea about tech stacks like rag model, vector db etc.	10/09/2025
Prototype/First Draft	Local & HuggingFace data Processing	17/09/2025
Mid-Term Review	Pinecone Integration & RAG model creation	03/10/2025
Final Submission	Full stack streamlit Application	15/10/2025
Presentation	Presenting the whole project	Will be communicated shortly

5b. Project execution details

The project was executed step by step by cloning, configuring, and running the application locally. The detailed process is as follows:

1. Clone the Repository

The complete source code for this project is hosted on GitHub. To begin, the repository was cloned using the following command:

```
git clone -b Monika_NV https://github.com/Springboard-Internship-2025/AI-Based-Legal-Reference-and-Case-Retrieval-System_September_2025.git
```

2. Install Dependencies

Navigate to the project directory and install the required dependencies by running:

```
pip install -r requirements.txt
```

This ensures that all necessary Python libraries such as streamlit, langchain, pinecone, huggingface, and dotenv are properly installed.

3. Set Up Environment Variables

Create a .env file in the root directory to store confidential API keys. The file should contain the following lines:

```
OPENAI_API_KEY="your_openai_api_key"  
PINECONE_API_KEY="your_pinecone_api_key"
```

These environment variables are used to connect the application securely with **OpenAI** for text generation and **Pinecone** for vector database operations.

1.

4. Verify Configuration

The configuration of environment variables and project paths is managed using the dotenv library. The task5.py file handles document chunking, embedding generation, and data upload to Pinecone.

The environment setup is verified by loading variables through:

```
from dotenv import load_dotenv
load_dotenv()
```

5. Run Data Embedding Script (task5.py)

The script task5.py performs the following operations:

Loads and processes legal documents from the data folder.

Splits them into smaller text chunks.

Generates embeddings using the Hugging Face model sentence-transformers/all-MiniLM-L6-v2.

Uploads these embeddings to a **Pinecone index** named legal-chatbot.

To execute this step, run:

```
python task5.py
```

6. Run the Main Application (app.py)

The main application file app.py initializes the Streamlit interface, handles authentication through auth.py, and connects to the Pinecone vector database. It also integrates the **Retrieval-Augmented Generation (RAG)** model using LangChain to answer legal queries.

To start the application, run the command:

```
streamlit run app.py
```

7. Project Workflow

User Authentication: Handled through auth.py, enabling users to sign up, log in, and manage their profiles.

Legal Query Processing: The system uses LangChain's RAG chain to retrieve and summarize relevant Indian Penal Code sections from the embedded dataset.

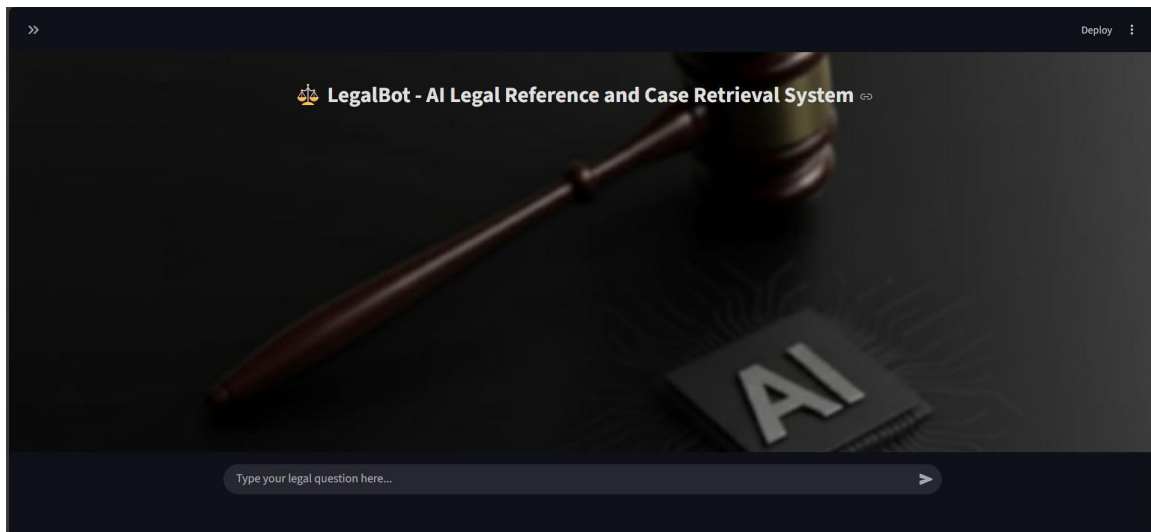
System Template: The system_template.py defines the structured prompt ensuring consistent, legally formatted AI responses.

8. Testing and Verification

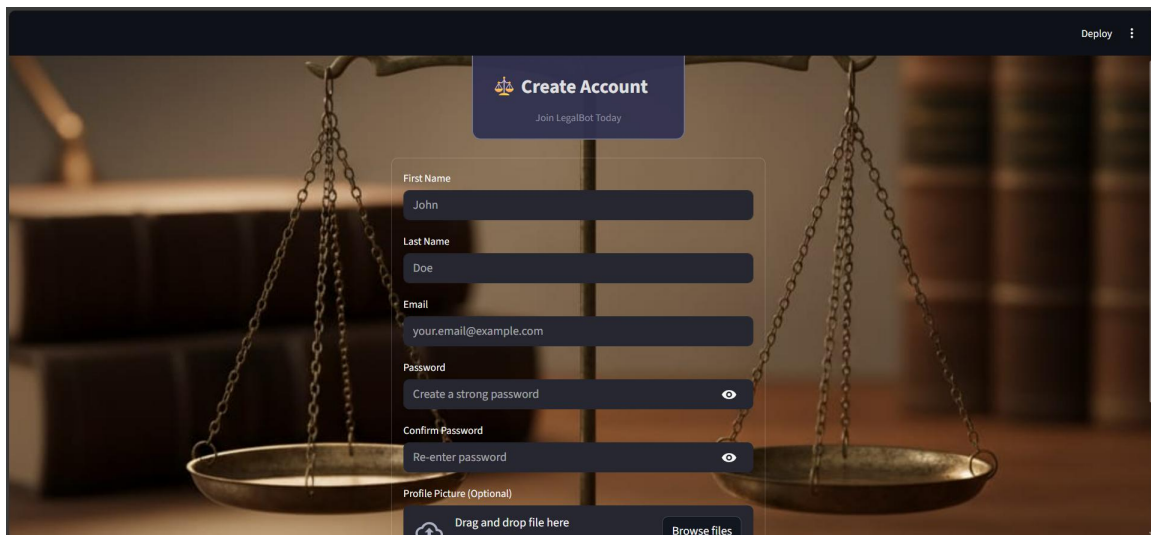
Once the application is live, users can enter legal queries into the chatbot interface. The responses are retrieved from the Pinecone vector store and generated by OpenAI's GPT model, providing accurate and structured legal references.

7. Snapshots / Screenshots

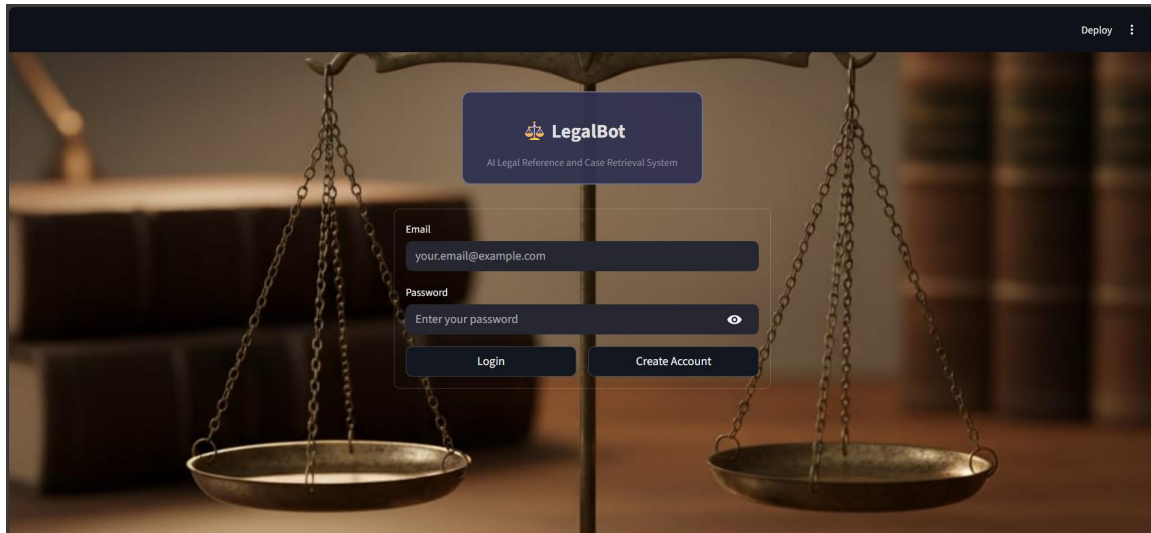
Home Page



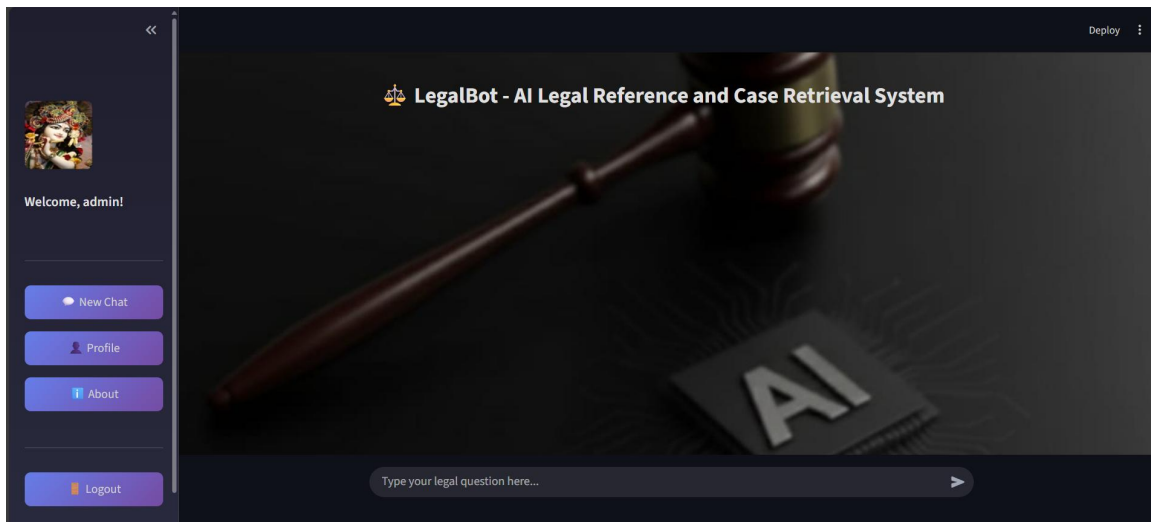
Sign in page



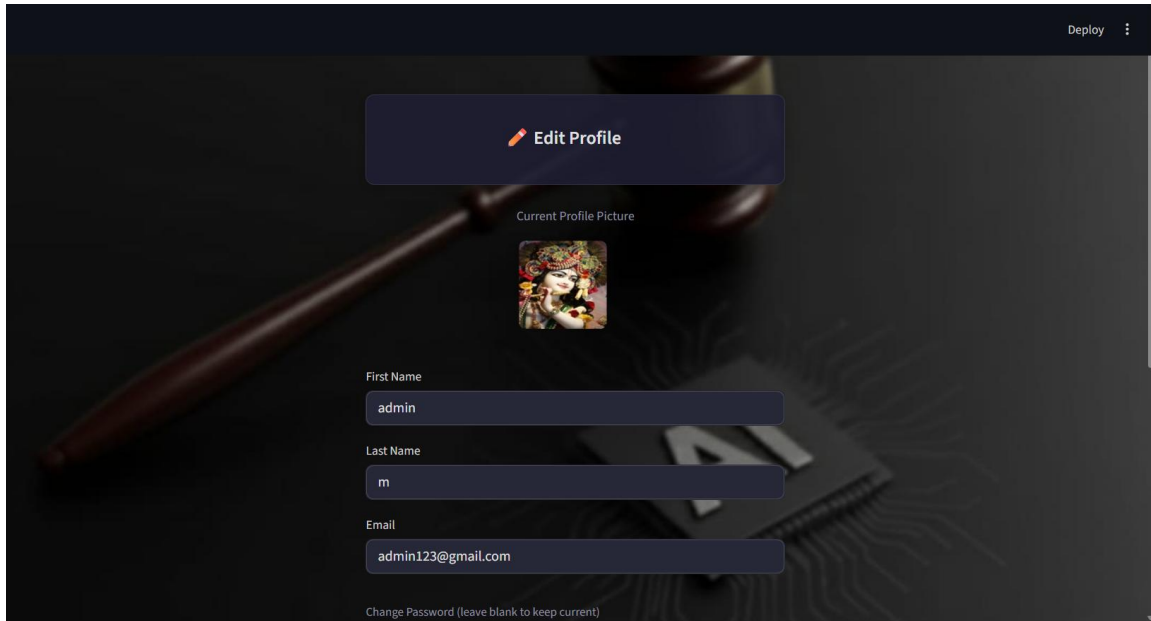
Login Page



Sidebar



Edit Profile



Deploy

 Edit Profile

Current Profile Picture



First Name

admin

Last Name

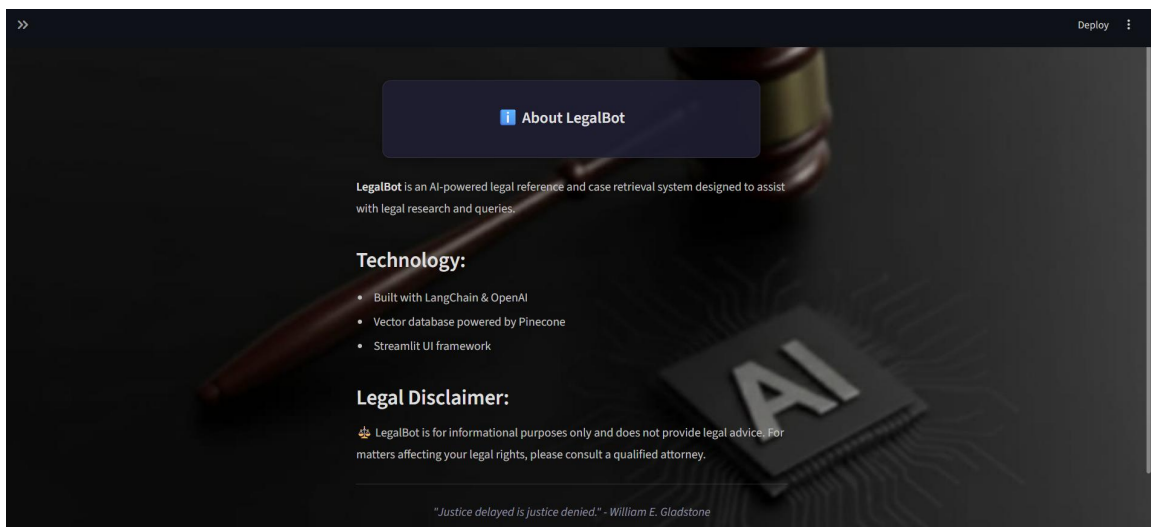
m

Email

admin123@gmail.com

Change Password (leave blank to keep current)

About Page



>>

Deploy

 About LegalBot

LegalBot is an AI-powered legal reference and case retrieval system designed to assist with legal research and queries.

Technology:

- Built with LangChain & OpenAI
- Vector database powered by Pinecone
- Streamlit UI framework

Legal Disclaimer:

 LegalBot is for informational purposes only and does not provide legal advice. For matters affecting your legal rights, please consult a qualified attorney.

"Justice delayed is justice denied." - William E. Gladstone

7. Challenges Faced

During the execution of the *AI-Based Legal Reference and Case Retrieval System* project, several challenges were encountered and resolved as described below:

1. Retriever Method Not Working Properly

While implementing the Retrieval-Augmented Generation (RAG) pipeline, the **retriever method** did not function as expected. The system failed to fetch relevant documents accurately using the retriever, even though the **similarity search method** was returning correct results.

Resolution: After debugging, it was found that the retriever configuration in LangChain was incompatible with the Pinecone index structure. To resolve this, the system temporarily used the similarity search method for retrieval until the retriever issue could be properly tuned with the correct embeddings and index settings.

2. Pinecone Index Connection Errors

At times, the system showed connection issues while uploading or fetching data from the Pinecone vector database due to incorrect environment variables or API key mismatches.

Resolution: The issue was fixed by regenerating and reconfiguring the API keys in the .env file and verifying that the correct Pinecone index name was being used.

3. Prompt Formatting Errors

The structured **SYSTEM_TEMPLATE** required strict formatting (legal disclaimer, provisions, summary, citations). Initially, missing line breaks or extra spaces caused response errors.

Resolution: The template was refined, and formatting was adjusted to ensure consistency across all generated responses.

8. Learnings & Skills Acquired

I learned both **technical skills** and **soft skills**:

1. Technical Skills:

Built web applications using **Python** and **Streamlit**.

Worked with **LangChain**, **HuggingFace embeddings**, and **Pinecone** for AI-based legal document retrieval.

Learned to integrate **APIs**, manage **databases (SQLite)**, and implement **user login and profiles**.

2. Domain Knowledge:

Understood how **legal research and case retrieval** works.

Learned about **Indian Penal Code provisions** and presenting legal information accurately.

Soft Skills:

Improved **problem-solving** by fixing retriever and search issues.

Enhanced **time management** and project organization.

9. Testimonials from team

1. The internship provided me with **hands-on experience in AI-based legal systems**.
2. I learned to implement and integrate **retriever and similarity search methods** effectively.
3. Working on this project improved my **coding, problem-solving, and debugging skills**.
4. I gained confidence in **using Streamlit, databases, and AI frameworks** for real-world applications.
5. Overall, this experience strengthened my **technical and teamwork skills** while contributing to a meaningful project.

10. Conclusion

The internship provided me with practical experience in building an **AI-based legal reference system**, strengthening both my technical and problem-solving skills. I gained hands-on knowledge in **Python, Streamlit, databases, and AI tools like LangChain and Pinecone**. Working on this project enhanced my confidence in **handling real-world challenges** such as retriever implementation and chat system integration. Overall, the internship was highly valuable, aligning well with my **academic learning and career goals in AI, software development, and legal technology**.

11. Acknowledgements

I would like to express my sincere gratitude to **Springboard Internship** for providing this valuable opportunity. I am thankful to my **mentor, Munikiran**, for their guidance, support, and timely feedback throughout the project. I also extend my appreciation to the **coordinator, Udayakumar**, and all team members who contributed to making this internship a successful and enriching experience.